

LABORATORY OBSERVATION/RECORD

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	1
Student Name	P. Govindamma
Roll No.	A24126552104
Date	30-08-2026

1. TITLE

Design a Static Webpage Using HTML Components

2. AIM

To design a static webpage using pure HTML5 components — such as headers, navigation, sections, articles, forms, tables, and lists — without using CSS, in order to understand semantic document structure.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the semantic HTML5 elements used to structure a webpage.
- To organize a webpage using header, nav, section, main, article, aside, and footer.
- To use a table for structural page layout.
- To create interactive form elements such as text, email, select, and textarea fields.
- To apply basic HTML formatting and text-level tags.
- To build a static, browser-renderable page without any external stylesheet.

4. THEORY

HTML (HyperText Markup Language) is the standard markup language used to create the structure and content of webpages. HTML5 introduced semantic elements that describe the meaning of the content they contain, rather than only its appearance. In this experiment, an entire webpage is built using raw HTML elements only, without any CSS, so that the browser's default rendering reveals the underlying document structure directly.

Header and Navigation

The <header> element represents the introductory section of the page, and typically contains the site title and a <nav> element, which groups the primary navigation links.

Sections, Main and Articles

The <section> element groups thematically related content, <main> represents the dominant content of the page, and <article> represents a self-contained, independently distributable piece of content such as a blog post.

Aside and Footer

The `<aside>` element holds content tangentially related to the main content, such as a sidebar. The `<footer>` element represents the closing section of the page, typically containing copyright information.

Tables for Layout

Although modern layouts use CSS, an HTML `<table>` can still be used to arrange content into rows and columns, as shown here for the header bar and the two-column body layout.

Forms and Input Elements

The `<form>` element collects user input using controls such as `<input>`, `<select>`, and `<textarea>`, grouped inside a `<fieldset>` with a `<legend>` as its caption.

The basic flow of the application is:

HTML Document → Semantic Structure (header/nav/section/main/article/aside/footer) → Browser Default Rendering → Static Webpage Output

5. ALGORITHM / PROCEDURE

- 1 Create a new HTML5 document with the DOCTYPE declaration.
- 2 Add the `<head>` section with character encoding and a page title.
- 3 Create a `<header>` containing a table with the site title and a navigation bar.
- 4 Add navigation links inside a `<nav>` element for Home, Articles, About, and Contact Us.
- 5 Insert a horizontal rule to visually separate the header from the page body.
- 6 Create a `<section>` for the hero area with a heading and introductory text.
- 7 Build the main body layout using a structural `<table>` with two columns.
- 8 In the left column, add a `<main>` element containing two `<article>` blocks.
- 9 Add headings, publish dates, and descriptive paragraphs to each article.
- 10 Add an interactive form inside the second article using `<fieldset>` and `<legend>`.
- 11 Add text, email, select, and textarea input controls to the form.
- 12 Add submit and reset buttons to the form.
- 13 In the right column, add an `<aside>` element with a benefits list and a components list.
- 14 Add a horizontal rule after the table to close off the main body.
- 15 Create a `<footer>` with a centered copyright notice.
- 16 Save the file as `index.html`.
- 17 Open the file in a web browser to render the page.
- 18 Verify the header, navigation, sections, form, sidebar, and footer display correctly.
- 19 Test the form fields, dropdown selection, and required-field validation.

6. PROGRAM

`index.html`

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Pure Semantic HTML Static Webpage</title>
</head>
<body>

  <!-- Header Section -->
  <header>
    <table width="100%" border="0" cellspacing="0" cellpadding="10">
      <tr>
        <td>
          <h1>StaticWeb</h1>
        </td>
        <td align="right">
          <nav>
            <a href="#home">Home</a> &nbsp;|&nbsp;&nbsp;
            <a href="#articles">Articles</a> &nbsp;|&nbsp;&nbsp;
            <a href="#about">About</a> &nbsp;|&nbsp;&nbsp;
            <a href="#contact">Contact Us</a>
          </nav>
        </td>
      </tr>
    </table>
  </header>

  <hr>

  <!-- Hero / Showcase Section -->
  <section id="home">
    <center>

```

```

      <h2>Semantic Structures Without CSS</h2>
      <p>This layout uses raw HTML components to build an organized document hierarchy that browsers can
        cleanly parse out-of-the-box.</p>
      <p><a href="#articles"><b>Read Our Articles Below &darr;</b></a></p>
    </center>
  </section>

  <hr>

  <!-- Main Content Layout using a Structural Table -->
  <table width="100%" border="0" cellspacing="0" cellpadding="20">
    <tr valign="top">
      <!-- Left Side: Main Content Column -->
      <td width="70%">
        <main id="articles">
          <article>
            <h2>1. Core Structural Tags</h2>
            <p>Published: July 7, 2026</p>
            <p>When you omit cascading style sheets entirely, the semantic markup handles all of the
              structural heavy lifting. Elements like <code>&lt;main&gt;</code>, <code>&lt;article&gt;</code>, and <code>&lt;section&gt;</code> tell screen readers and web scrapers exactly
              how information flows.</p>
            <p>Using raw browser defaults ensures excellent loading performance, perfect accessibility
              compliance, and absolute structural clarity.</p>
          </article>

```



```

        </ul>

        <h4>Required Components:</h4>
        <ol>
            <li>Document Type Definition</li>
            <li>Semantic Wrapper Blocks</li>
            <li>Data Input Control Interfaces</li>
        </ol>
    </aside>
</td>

</tr>
</table>

<hr>

<!-- Footer Section -->
<footer>
    <center>
        <p>&copy; 2026 Pure HTML Architecture. Assembled completely without style configurations.</p>
    </center>
</footer>

</body>
</html>
```

7. CODE EXPLANATION

Code	Explanation
<header>	Represents the introductory section of the page, holding the title and navigation.
<nav>	Groups the primary set of navigation links for the site.
<table>	Used here for structural layout: the header bar and the two-column body.
<hr>	Draws a horizontal rule to visually separate page sections.
<section>	Groups thematically related content, such as the hero/home area.
<center>	Centers its contents horizontally on the page.
<main>	Represents the dominant, unique content of the page body.
<article>	Represents a self-contained block of content, such as a post.
<code>	Marks up inline text as a code fragment.
<form>	Collects user input and defines how it is submitted.
<fieldset> / <legend>	Groups related form controls and gives the group a caption.
<input>	Creates form controls such as text, email, submit, and reset fields.
<select> / <option>	Creates a dropdown list of selectable choices.
<textarea>	Creates a multi-line text input control.
<aside>	Represents content tangentially related to the main content (sidebar).

Code	Explanation
/ /	Create unordered and ordered lists of items.
<footer>	Represents the closing section of the page, such as copyright text.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Open index.html in a browser	Webpage should load and display all sections	Page displayed correctly	Pass
TC-02	Check header area	Site title "StaticWeb" and navigation links should appear	Header displayed correctly	Pass
TC-03	Click a navigation link	Page should scroll to the corresponding section anchor	Scrolled to correct section	Pass
TC-04	Check hero section	Heading and introductory text should be centered	Displayed correctly	Pass
TC-05	Check article content	Both articles should display heading, date, and text	Displayed correctly	Pass
TC-06	Submit the form with empty Name field	Browser should require the Name field	Validation message shown	Pass
TC-07	Select a value from Inquiry Topic dropdown	Selected option should update the field	Dropdown updated correctly	Pass
TC-08	Type text into Message Details textarea	Text should appear inside the textarea	Text entered successfully	Pass
TC-09	Click "Clear Entries" button	All form fields should be reset	Fields cleared successfully	Pass
TC-10	Check sidebar and footer	Sidebar lists and footer copyright should display	Displayed correctly	Pass

9. OUTPUT

Each test case was executed in the browser individually and the rendered/observed output was captured below, corresponding to the test cases in Section 8.

Output 1 (TC-01): Open index.html in a browser

Semantic Structures Without CSS

This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.

[Read Our Articles Below](#)

1. Core Structural Tags

Published: July 7, 2026

When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like <main>, <article>, and <section> tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

General Feedback

Message Details:

Type your notes here...

Submit Form Data

Clear Entries

Document References

Static pages map data out directly without real-time database queries or processor overhead.

Core Benefits:

- Instant browser painting
- Low memory footprints
- High data compatibility

Required Components:

- Document Type Definition
- Semantic Wrapper Blocks
- Data Input Control Interfaces

Output 2 (TC-02): Check header area

Output 3 (TC-03): Click a navigation link

1. Core Structural Tags

Published: July 7, 2026

When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like <main>, <article>, and <section> tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

General Feedback

Message Details:

Type your notes here...

Submit Form Data

Clear Entries

Document References

Static pages map data out directly without real-time database queries or processor overhead.

Core Benefits:

- Instant browser painting
- Low memory footprints
- High data compatibility

Required Components:

- Document Type Definition
- Semantic Wrapper Blocks
- Data Input Control Interfaces

Output 4 (TC-04): Check hero section

Semantic Structures Without CSS

This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.

[Read Our Articles Below ↴](#)

Output 5 (TC-05): Check article content

1. Core Structural Tags

Published: July 7, 2026

When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like `<main>`, `<article>`, and `<section>` tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Output 6 (TC-06): Submit the form with empty Name field

User Inquiry Form

Full Name:

Email Address:

Please fill out this field.

Inquiry Topic:

Message Details:

This is a sample inquiry message typed into the textarea for testing.

Output 7 (TC-07): Select a value from Inquiry Topic dropdown

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

Output 8 (TC-08): Type text into Message Details textarea

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

Message Details:

This is a sample inquiry message typed into the textarea for testing.

Output 9 (TC-09): Click "Clear Entries" button

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

Message Details:

Output 10 (TC-10): Check sidebar and footer

Document References

Static pages map data out directly without real-time database queries or processor overhead.

Core Benefits:

- Instant browser painting
- Low memory footprints
- High data compatibility

Required Components:

1. Document Type Definition
2. Semantic Wrapper Blocks
3. Data Input Control Interfaces

Fig 9.10a: The sidebar displays the Document References heading, the Core Benefits list, and the Required Components list.

© 2026 Pure HTML Architecture. Assembled completely without style configurations.

Fig 9.10b: The footer displays the centered copyright notice.

11. RESULT

Thus, a static webpage was successfully designed using pure HTML5 components, including header, navigation, section, main, article, aside, and footer elements, along with a table-based layout and an interactive form. The page rendered correctly in the browser using only default styling, and all test cases relating to structure, navigation, and form behaviour were verified successfully, with individual outputs captured for each test case above.